

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



Deep Learning

Introducción a Deep Learning

Ariel Reyes Pardo
Dpto. Ciencia de la Computación

Curso cubre los tópicos básicos/fundamentales de Deep Learning

Fecha	Tema
07/07	Introducción a DL
21/07	CNN
28/07	Aplicaciones de CNN
04/08	C1 y T1
11/08	RNN
18/08	Modelos de lenguaje
25/08	Foundation Models
01/09	C2 y T2

Reconocimiento Visual

Procesamiento de Lenguaje Natural

Evaluaciones: 2 Controles y 2 Tareas

- Tendremos 2 controles individuales, al inicio de las sesiones 4 y 8, con un valor de 25% de la nota final, cada uno.
- Tendremos 2 tarea en parejas, con un valor de 25% de la nota final, cada una.
- Enunciados de las tareas se publicarán los días 29/07 y 26/08.
- Las tareas se entregarán los días 07/08 y 05/09.

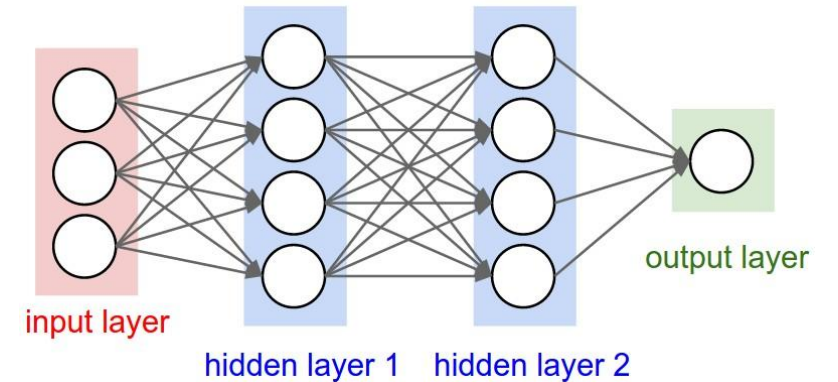
Fecha	Tema
07/07	Introducción a DL
21/07	CNN
28/07	Aplicaciones de CNN
04/08	C1 y T1
11/08	RNN
18/08	Modelos de lenguaje
25/08	Foundation Models
01/09	C2 y T2

Buscamos un mecanismo de aprendizaje, suficientemente general, para aprender aspectos complejos y eventualmente desconocidos de los datos, con el fin de mejorar el rendimiento en las tareas

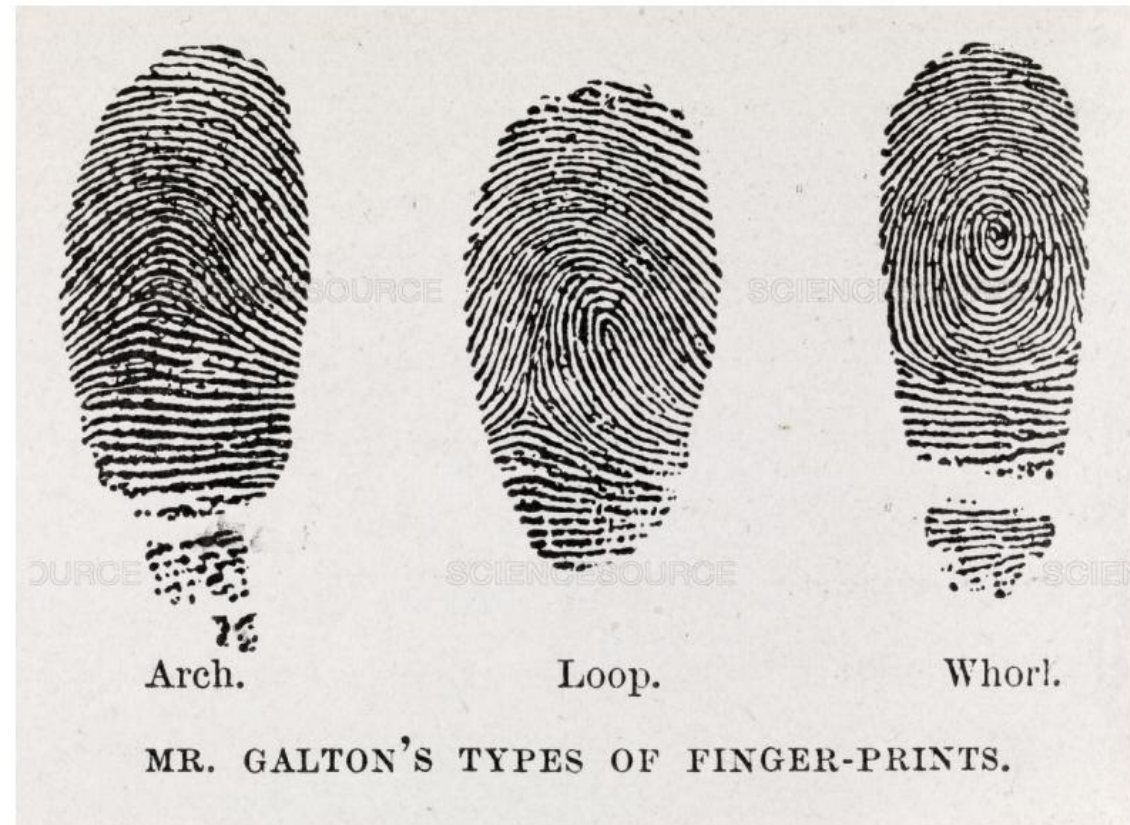


Comencemos con la materia propiamente tal

- Para lograr este objetivo, necesitamos discutir dos elementos fundamentales:
 1. Aprendizaje de *features*
 2. Representación composicional jerárquica
- Revisaremos estos puntos en detalle, antes de llegar al primer modelo que discutiremos



El concepto de *feature* ha sido clave para el avance de ML



El concepto de *feature* ha sido clave para el avance de ML



Figure 6-11. Some minutiae patterns used to analyze fingerprints.

Name	Visual Appearance
1. Ending ridge (including broken ridge)	1. [Diagram of a ridge ending]
2. Fork (or bifurcation)	2. [Diagram of a ridge bifurcating]
3. Island ridge (or short ridge)	3. [Diagram of a short ridge]
4. Dot (of very short ridge)	4. [Diagram of a dot]
5. Bridge	5. [Diagram of a bridge]
6. Spur (or hook)	6. [Diagram of a spur]
7. Eye (enclosure or island)	7. [Diagram of an eye]
8. Double bifurcation	8. [Diagram of a double bifurcation]
9. Delta	9. [Diagram of a delta]
10. Trifurcation	10. [Diagram of a trifurcation]

Durante mucho tiempo, el esquema dominante fue el de *features* hechas a mano (*handcrafted features* o *feature engineering*)

d1 : Mary loves Movies, Cinema and Art

d2 : John went to the Football game

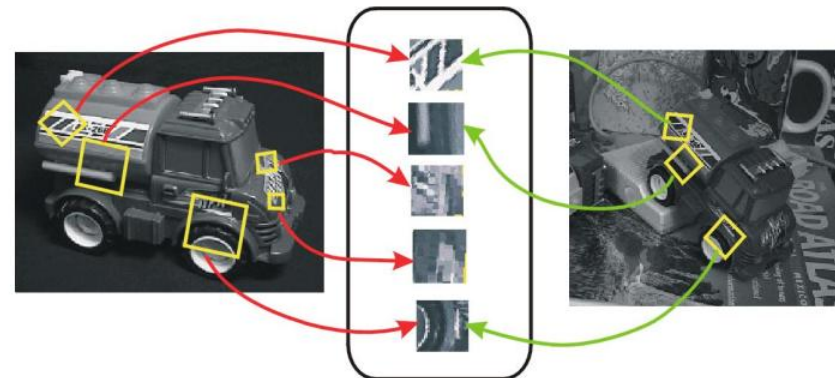
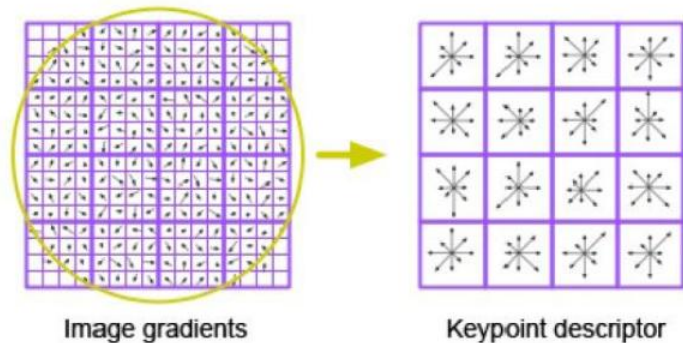
d3 : Robert went for the Movie Delicatessen

Class 1 : Arts

Class 2 : Sports

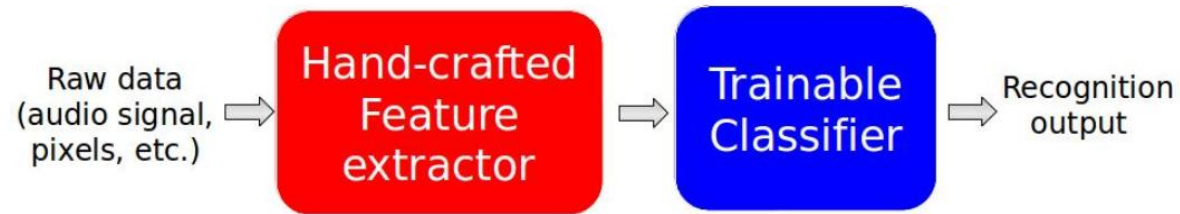
Class : Arts

	Mary	Loves	Movies	Cinema	Art	John	Went	to	the	Delicatessen	Robert	Football	Game	and	for
d1	1	1	1	1	1									1	
d2						1	1	1	1			1	1		
d3			1				1		1		1				1

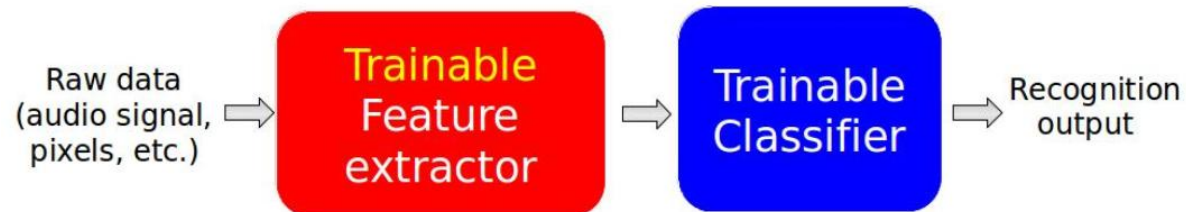


Enfoques modernos presentan un cambio de paradigma respecto a la extracción de *features*

- El enfoque tradicional de reconocimiento de patrones se ha basado por 50 años en *features* hechas a mano:

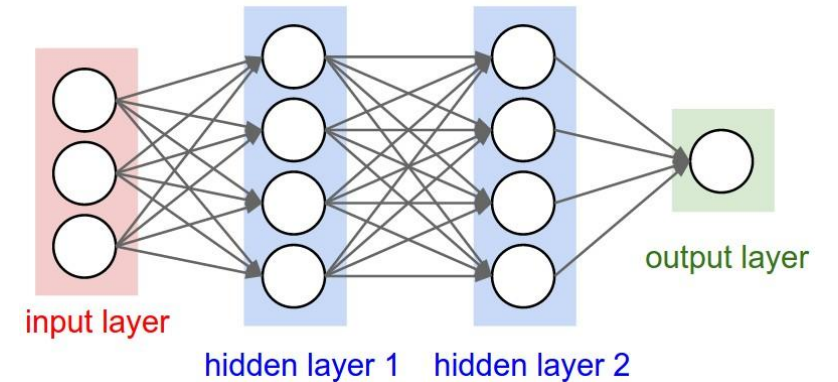


- El enfoque moderno se basa en el aprendizaje de *features*:



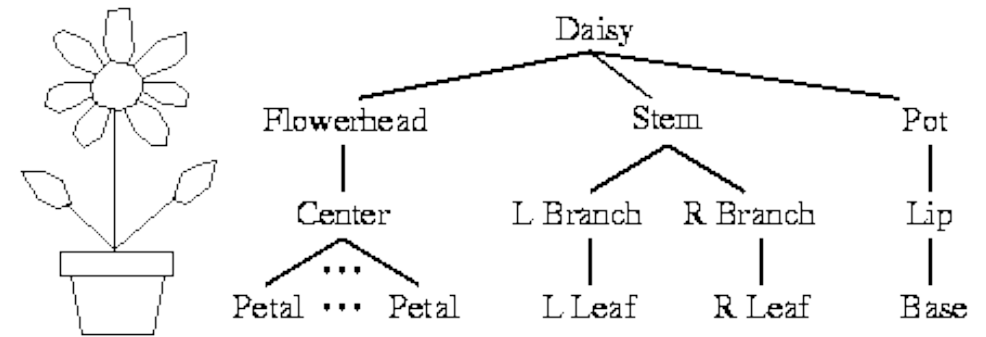
Comencemos con la materia propiamente tal

- Para lograr este objetivo, necesitamos discutir dos elementos fundamentales:
 1. Aprendizaje de *features*
 2. Representación composicional jerárquica
- Revisaremos estos puntos en detalle, antes de llegar al primer modelo que discutiremos.



Representaciones composicionales y jerárquicas son muy comunes en la naturaleza

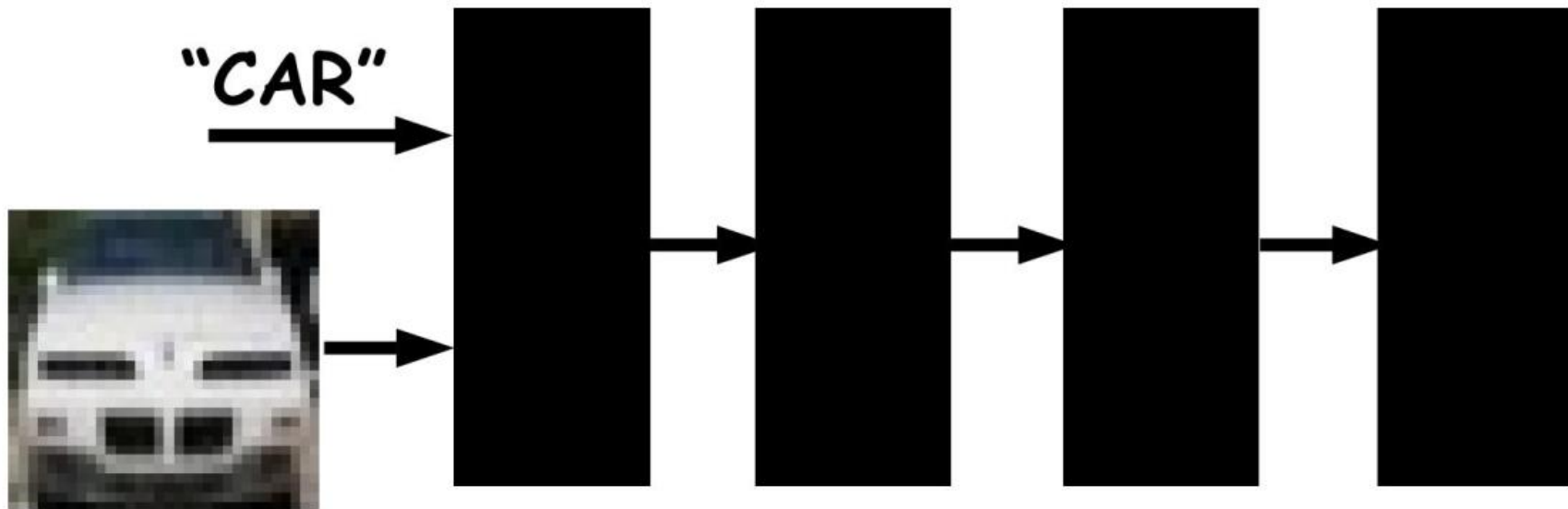
- Se organizan desde niveles bajos a altos de abstracción.
- La composicionalidad es su principal característica.
- Permiten describir infinitas configuraciones a partir de un número acotado de elementos.
- Algunos ejemplos:
 - Sonidos -> fonemas -> sílabas -> palabras
 - **Píxeles -> bordes -> partes -> objetos -> escenas**
 - Caracteres -> palabras -> grupos -> cláusulas -> frases



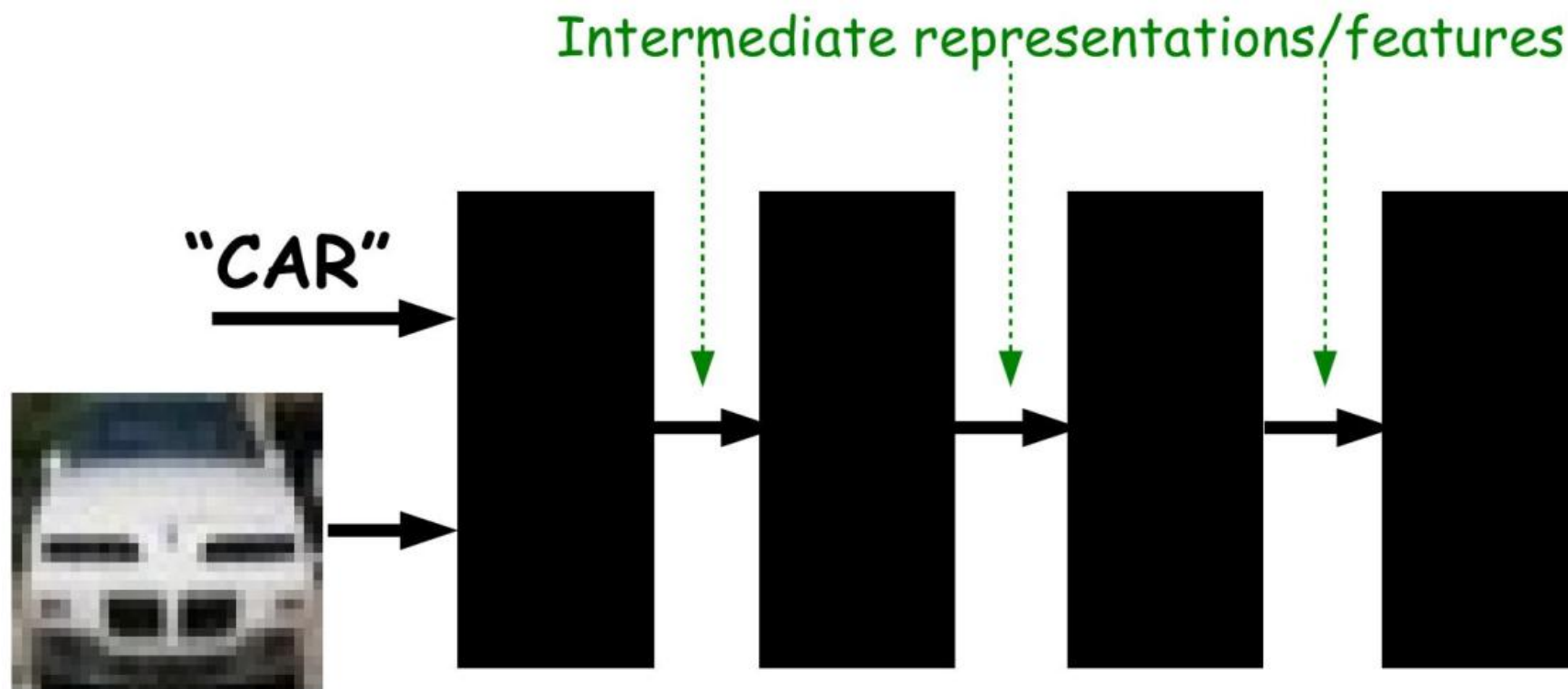
En visión, en particular, estas representaciones son muy intuitivas



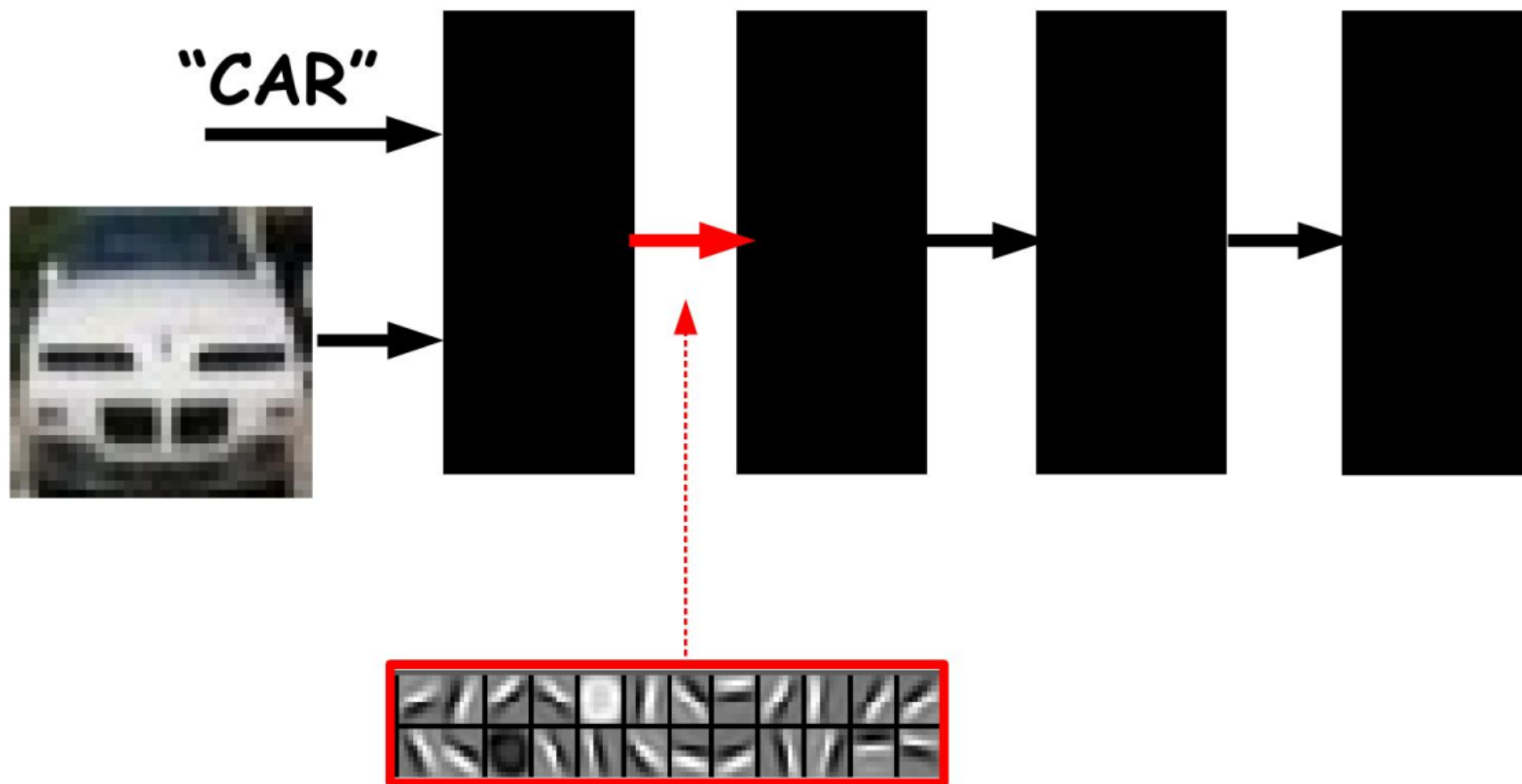
En visión, en particular, estas representaciones son muy intuitivas



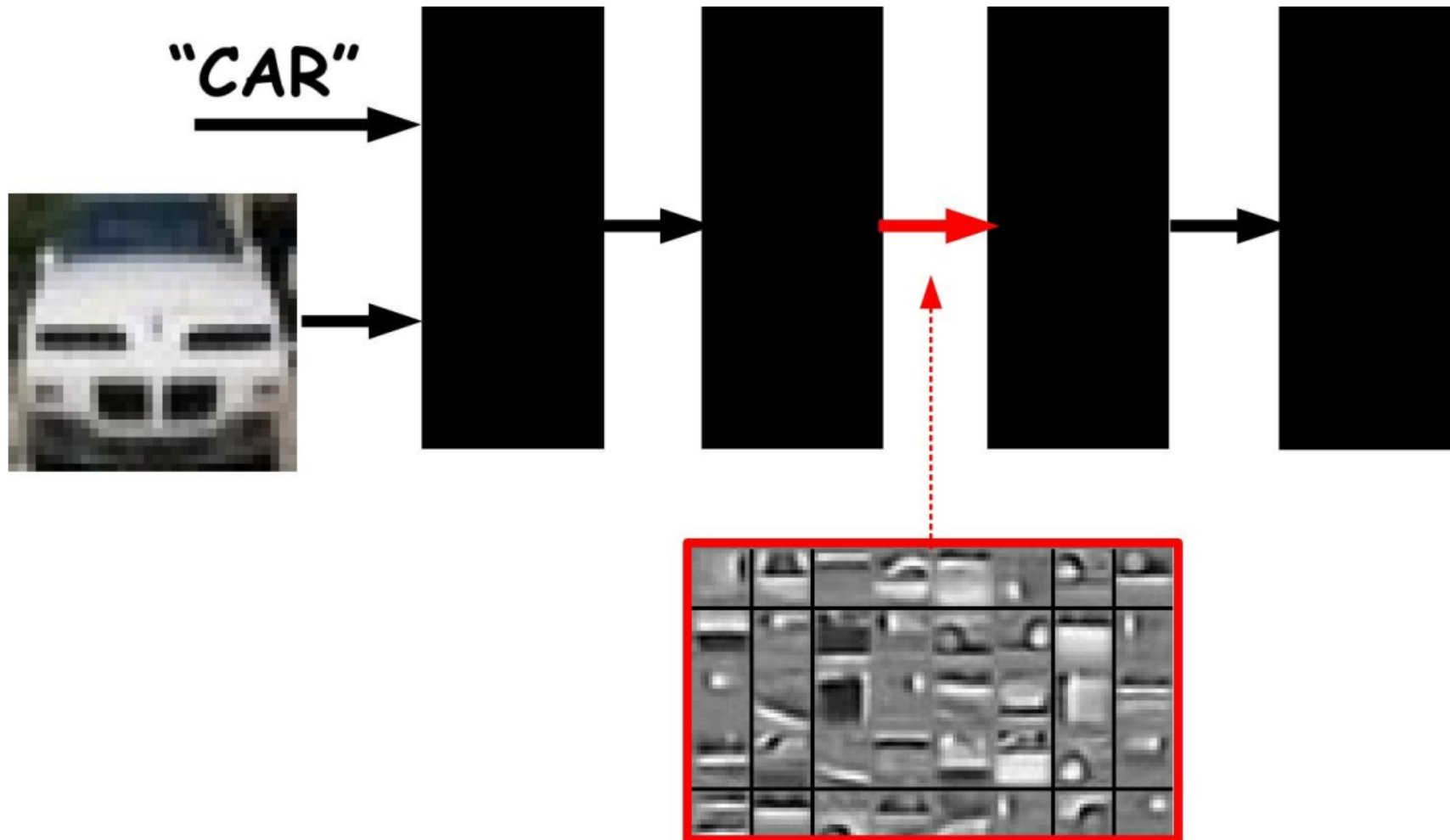
En visión, en particular, estas representaciones son muy intuitivas



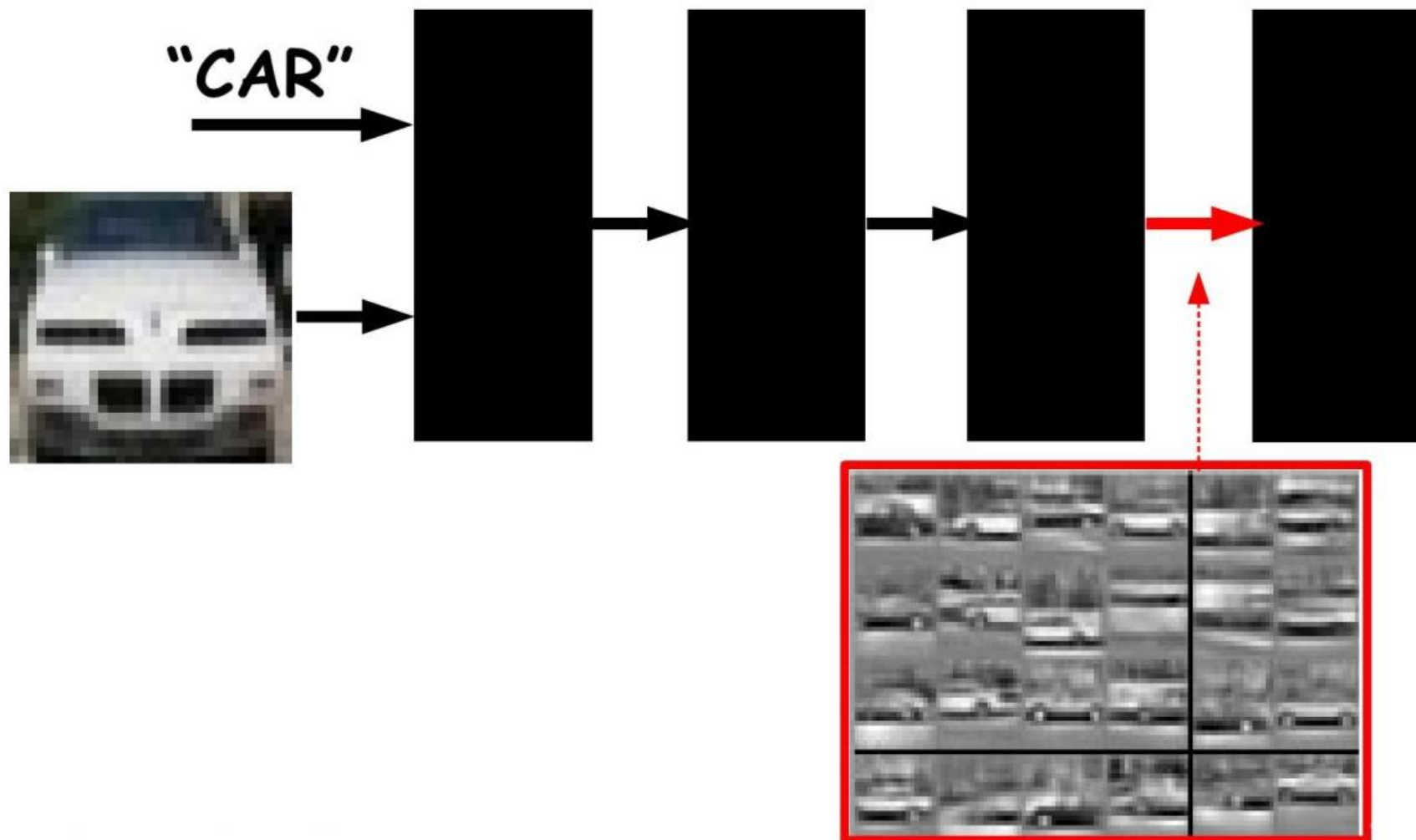
En visión, en particular, estas representaciones son muy intuitivas



En visión, en particular, estas representaciones son muy intuitivas



En visión, en particular, estas representaciones son muy intuitivas

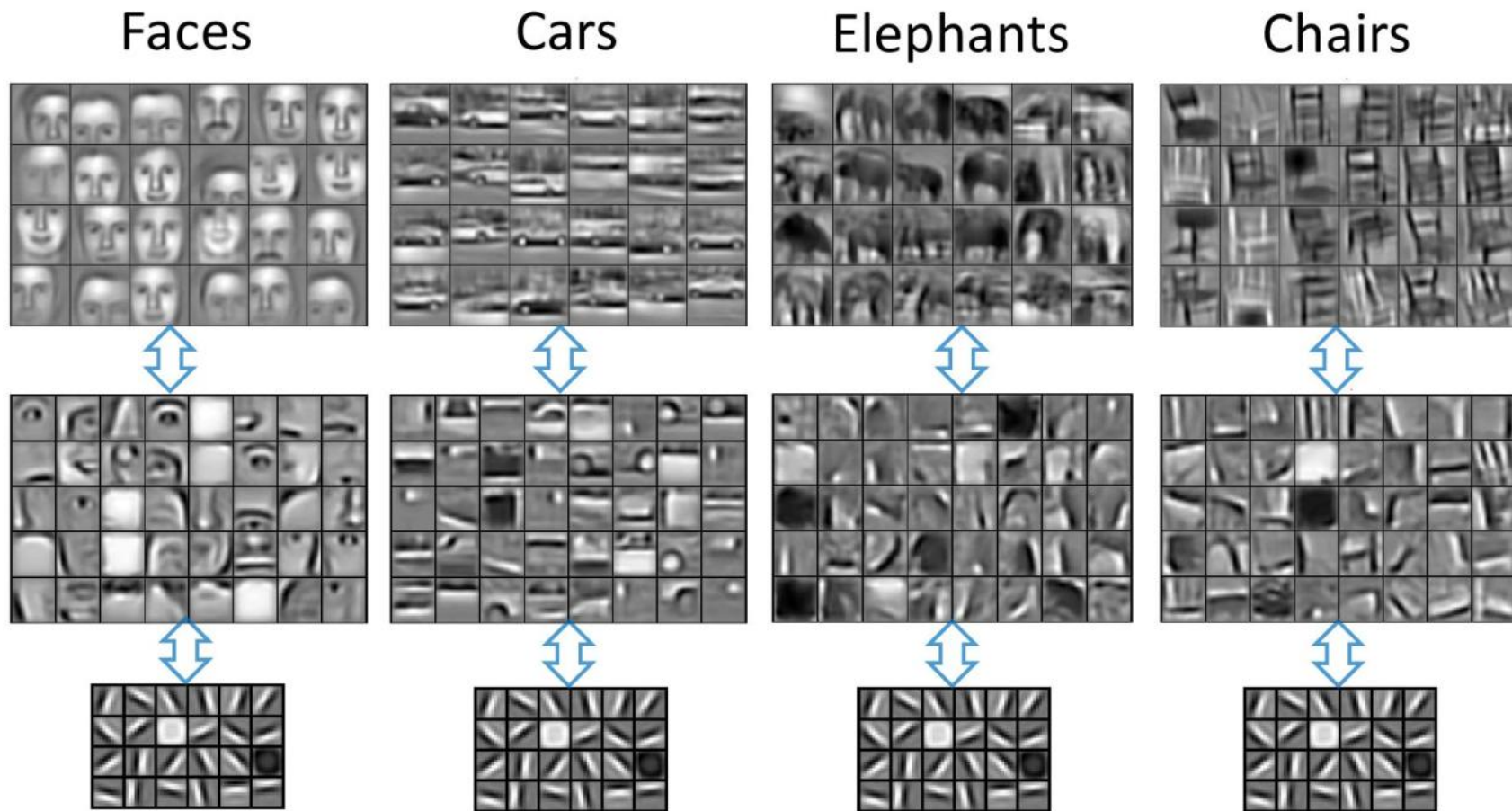


¿Qué se aprende en los distintos niveles?

- Capas de bajo nivel (poca profundidad) detectan *features* genéricas y locales que funcionan como bloques de construcción: fonemas y bordes en reconocimiento de voz y visual, respectivamente.
- Capas de alto nivel detectan *features* especializadas y globales, que son más robustas a variaciones: independientes del que habla o del punto de vista, en reconocimiento de voz y visual, respectivamente.



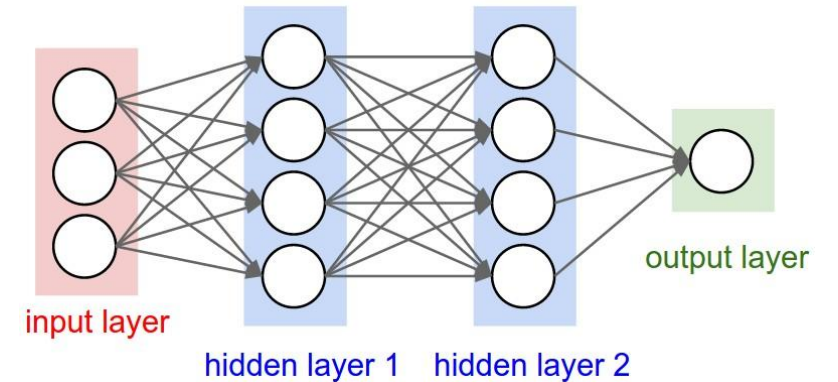
¿Qué se aprende en los distintos niveles?

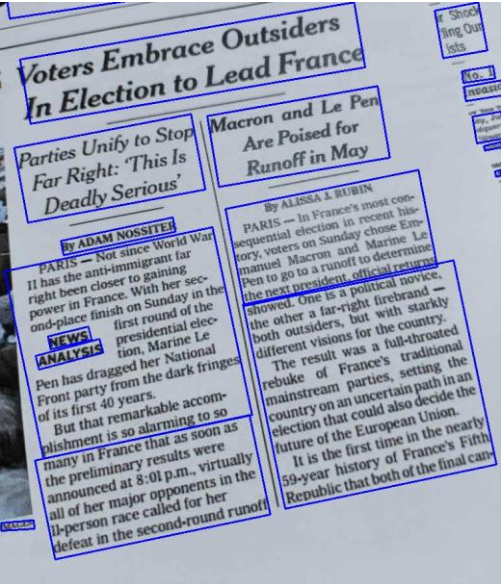
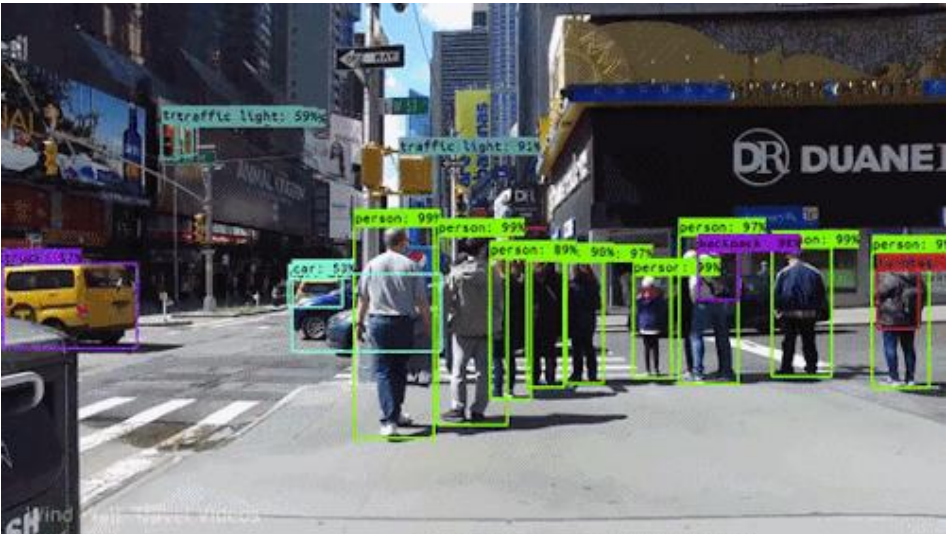


Ya tenemos todo para introducir correctamente a Deep Learning

Deep Learning satisface lo buscado

- Modelos altamente generales, que **aprendan** capas de presentaciones composicionales.
- Capas son formadas por unidades diferenciables entrenadas en conjunto (*differentiable programming*) en base a una función objetivo común.
- Al menos un nivel de representaciones intermedias, pero generalmente son muchas, para reducir complejidad (profundidad por sobre amplitud).
- Actualmente es el mecanismo de aprendizaje más exitoso, manteniendo récords en gran cantidad de tareas complejas, como reconocimiento de voz, caracteres, objetos, etc.



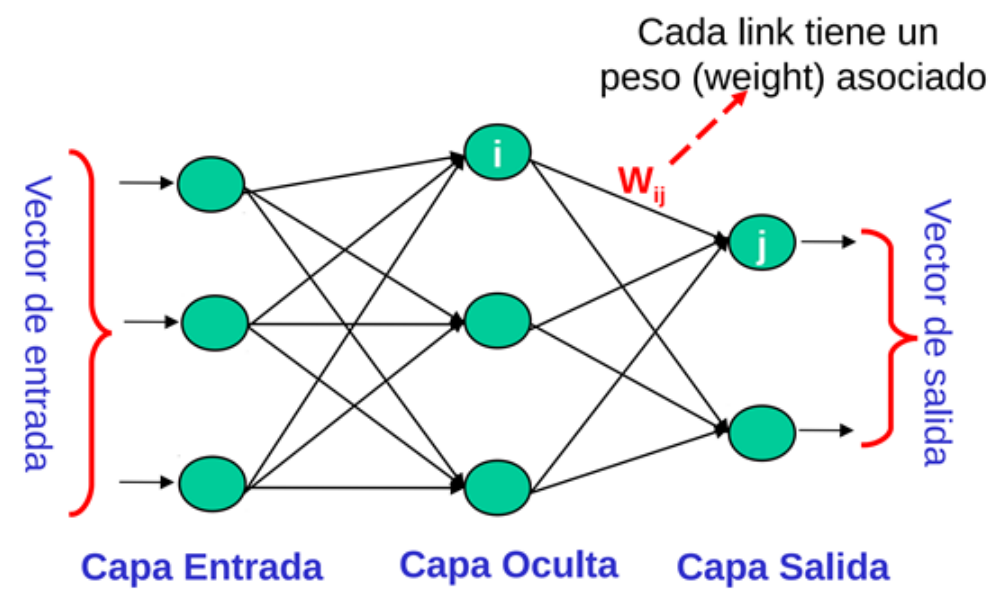


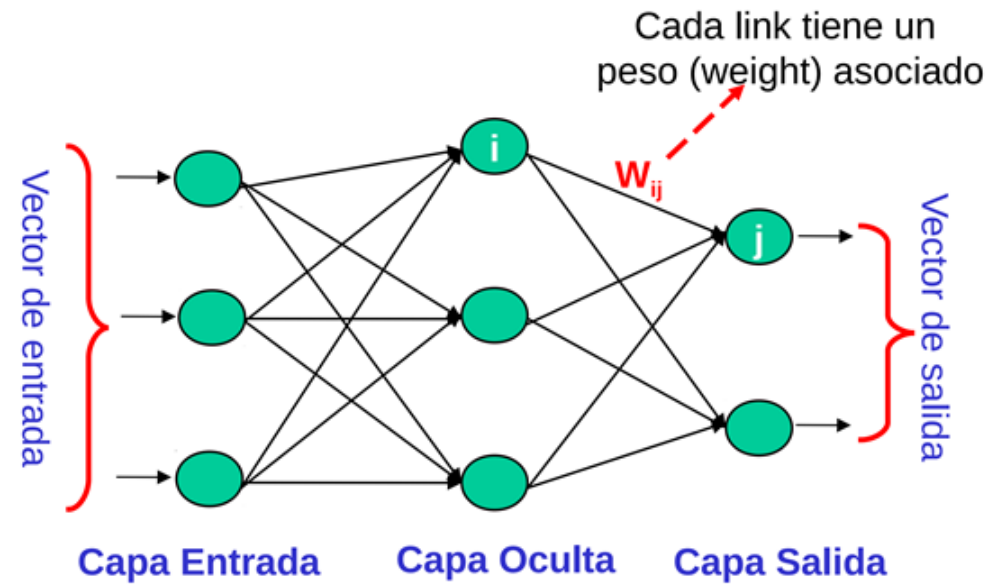
Generated Question	Answer
How many bears are laying on the ice?	"two"
What are the two animals laying on the ice?	"bears"
What are the bears doing?	"laying down"
Two bears are laying down on what?	"ice"
Where are the bears laying?	"on the ice"
Are the bears on the ice?	"yes"



A dog wearing a Superhero outfit with red cape flying through the sky

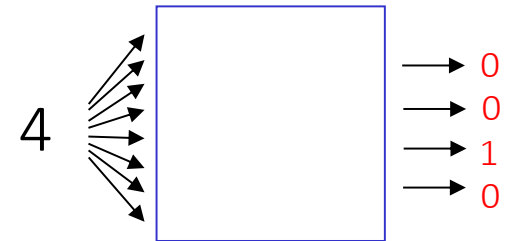
Ya estamos listos para ver el primer modelo de Deep Learning,
propiamente tal: los multi-layer perceptron (feed-forward network)

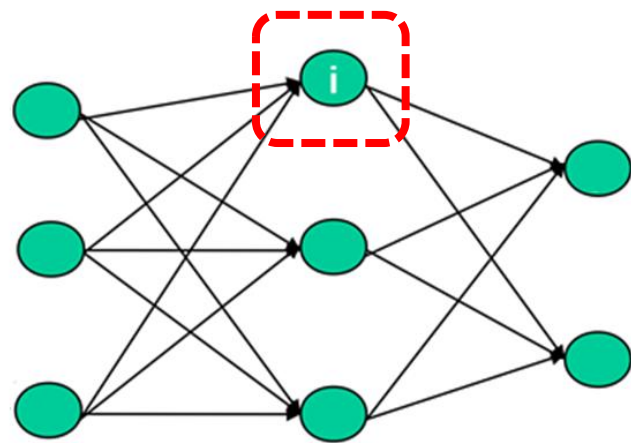




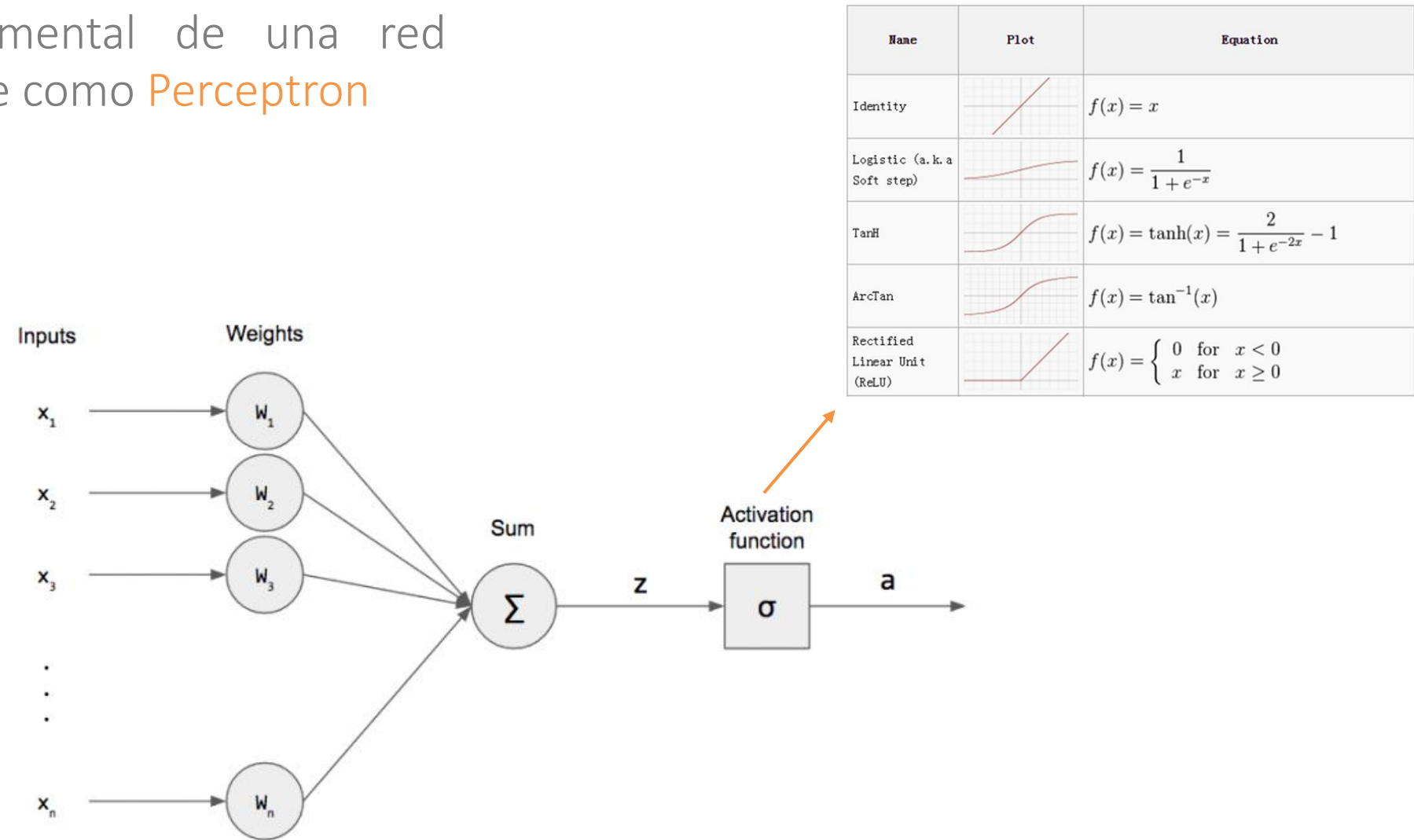
Dada la estructura de la red, ¿cómo podemos ajustar los pesos para que al ingresar un vector de entrada determinado obtengamos la salida deseada?

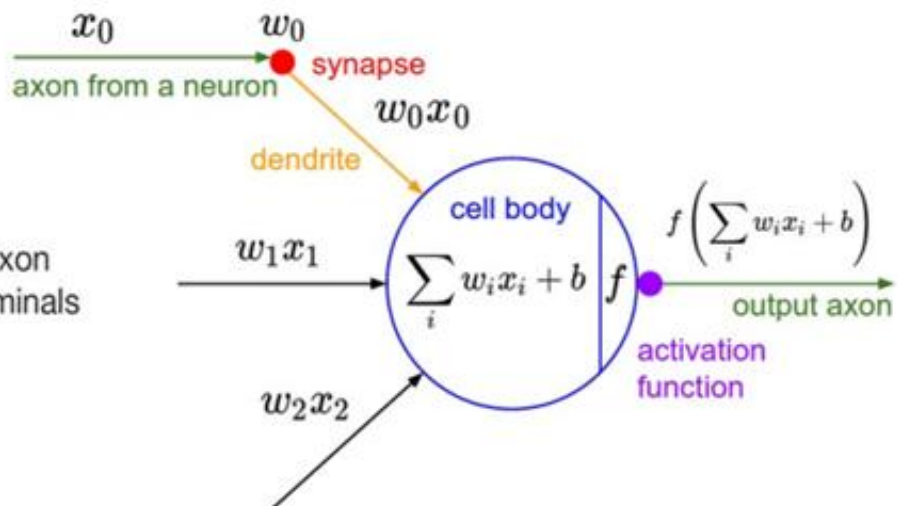
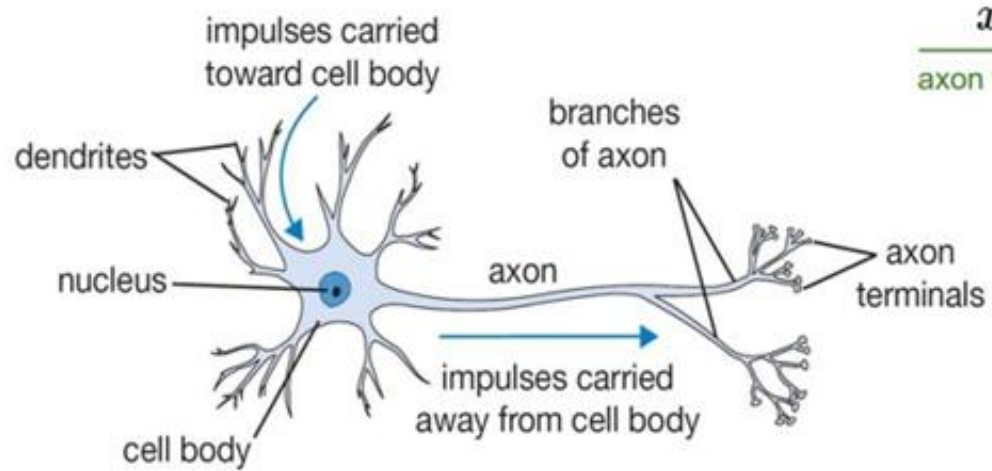
Ejemplo: cuando la red recibe una imagen de un 4, su codificación de salida debe indicar un 4.





La unidad fundamental de una red neuronal se conoce como **Perceptron**



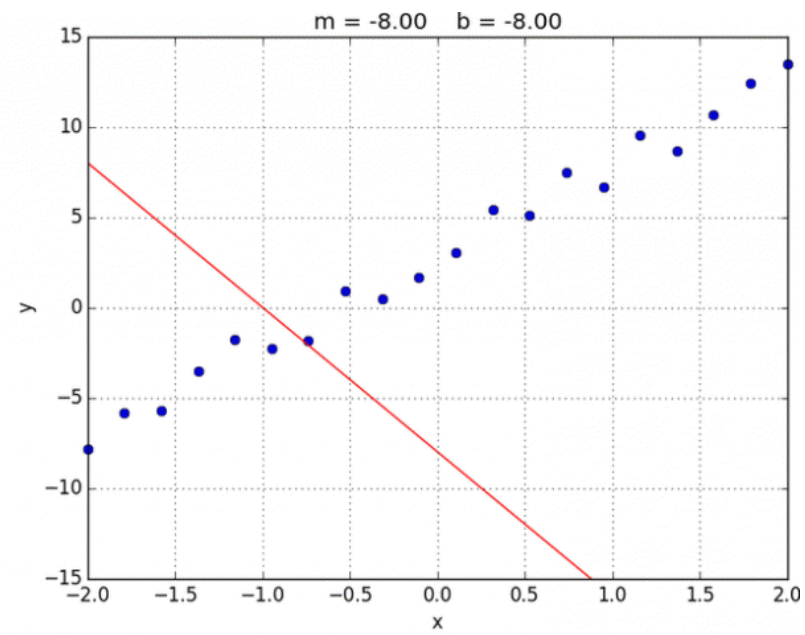
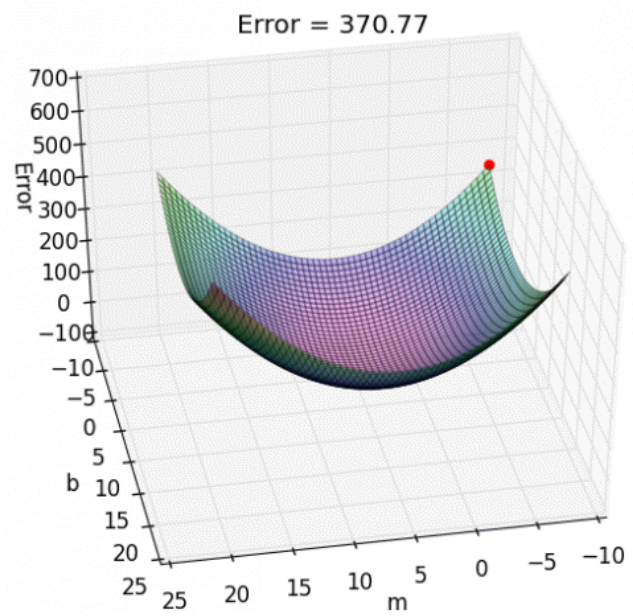


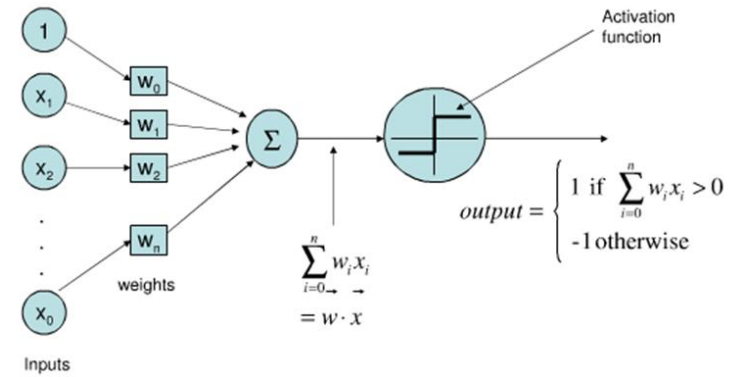
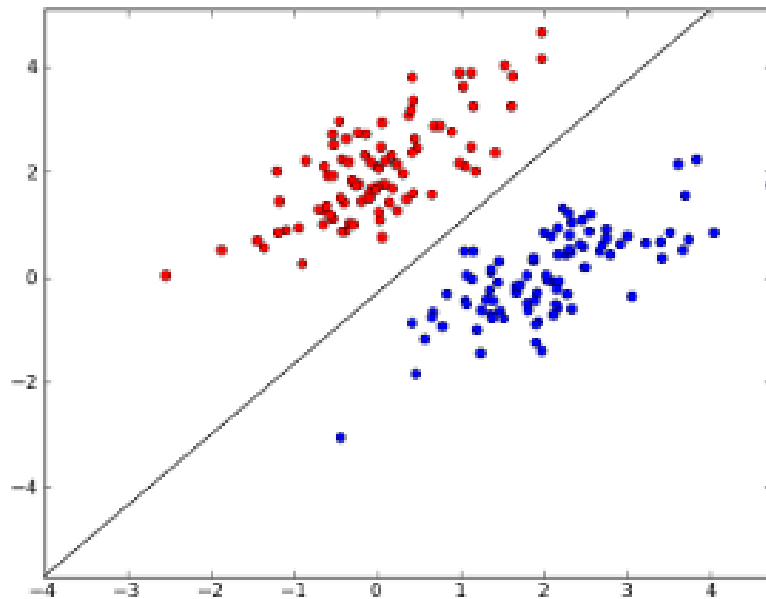
Un **Perceptron** permite estimar funciones de manera supervisada

- Si tenemos suficientes datos, pares (x_i, y_i) , es posible construir una función que nos indique cuán buena es en promedio la estimación del Perceptron.
- Sea $f(x; w) = \sigma(\sum_{i=1}^m w_{(i)} x_{(i)}) = \sigma(w \cdot x)$, una función que modela la aplicación de un Perceptrón parametrizado por un vector de pesos w , a un vector de características x .
- Sin suponer cosas raras sobre los datos, una posible **función de pérdida** (o costo, o error) puede quedar de la siguiente manera:

$$J(w) = \frac{1}{2n} \sum_{i=1}^n (f(x_i; w) - y_i)^2$$

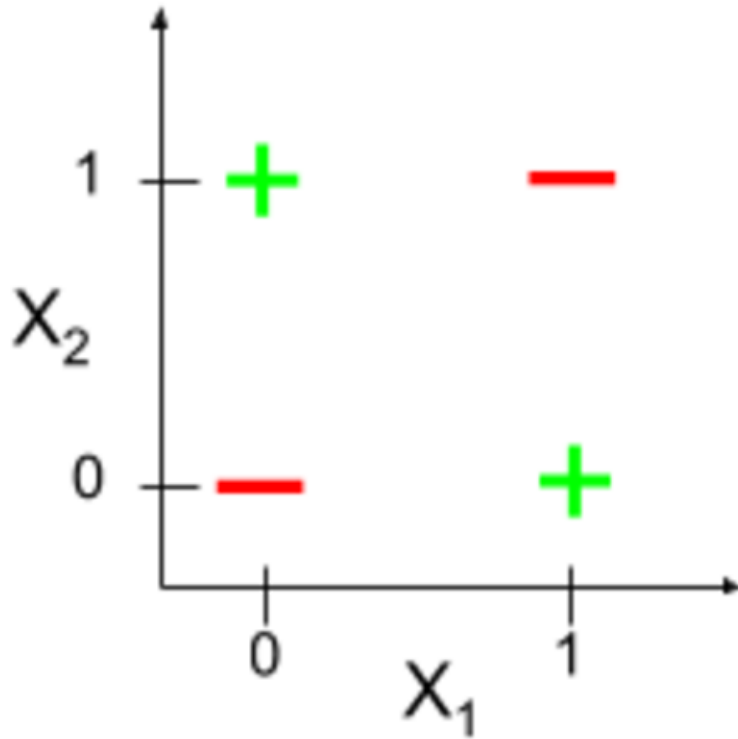
- ¿Qué representa el valor mínimo de esta función, con respecto a los pesos w ?





Perceptron puede clasificar problemas linealmente separables

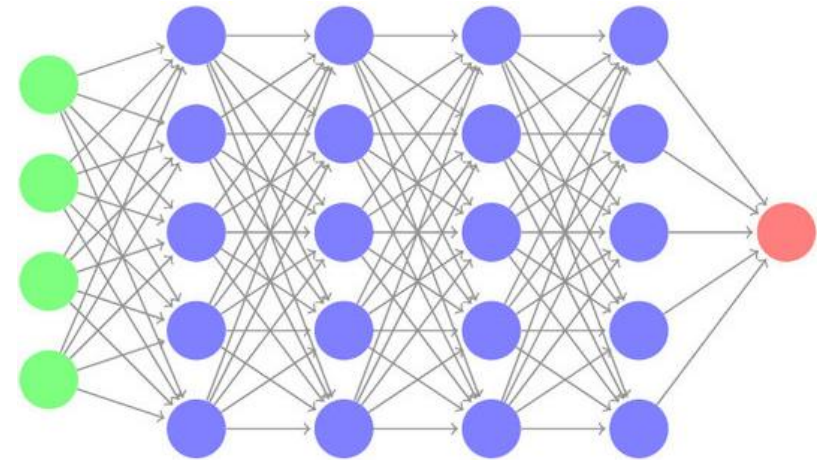
Tenemos entonces el mismo problema que con los SVM con kernel lineal

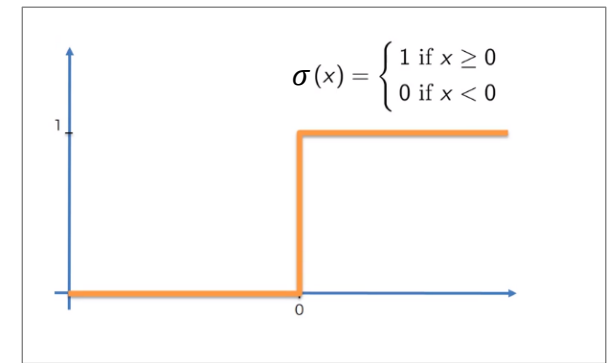
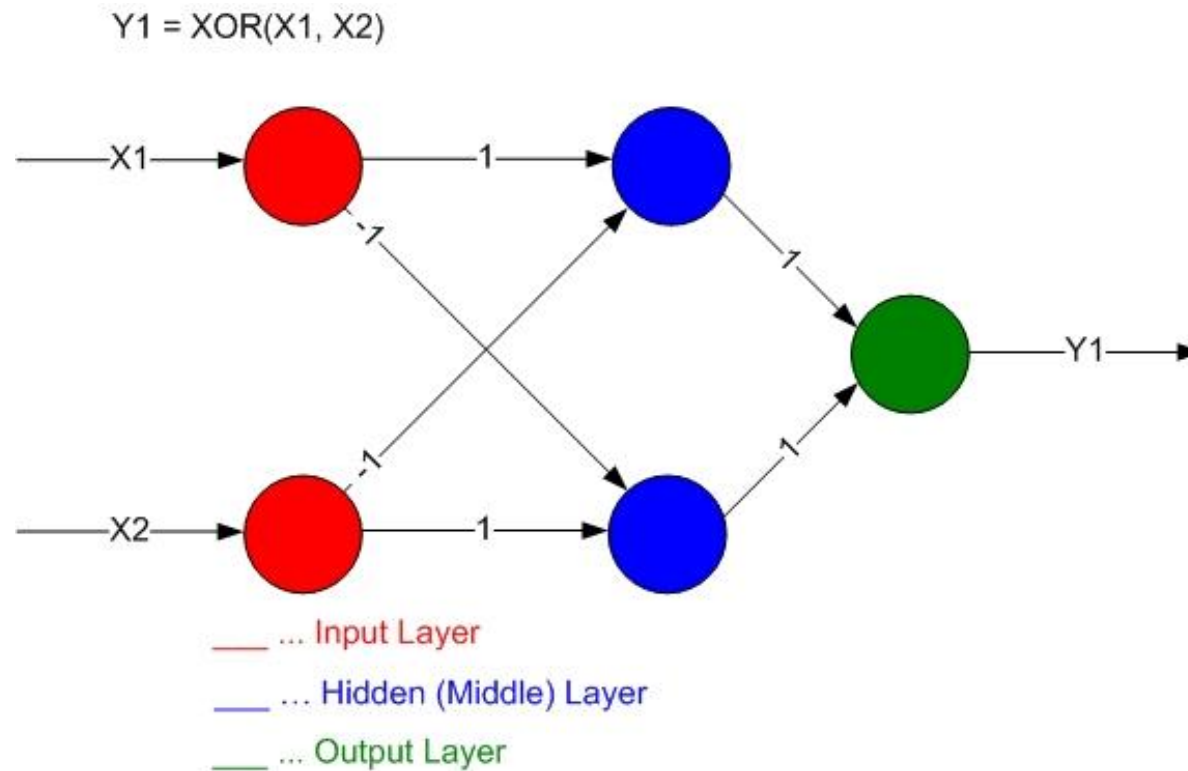


x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0

MLP generaliza el concepto de Perceptron

- Un Multilayer Perceptron (MLP), combina múltiples perceptrones en múltiples capas. Es conocido también como Deep Feed Forward Network (DFF).
- ¿Permite esta estructura resolver problemas más complejos que los que puede resolver un Perceptron?





Al agregar una nueva capa (oculta), podemos representar **conceptos intermedios** entre el input y el output.

Sin glamour, ¿qué es en realidad un MLP?

Siempre es bueno pensar también en las redes neuronales como una composición de funciones:

- MLP de 1 capa (Perceptron):

$$f = \sigma(W_1 x)$$

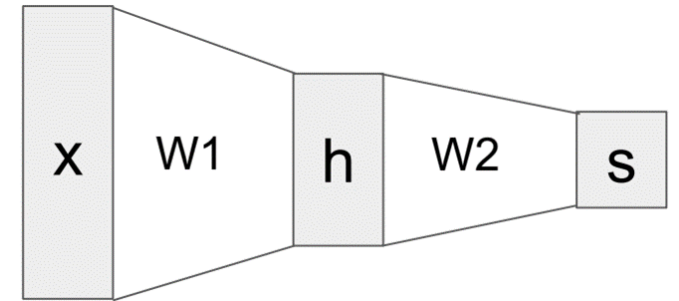
* $\sigma(z)$ = función de activación

- MLP de 2 capas:

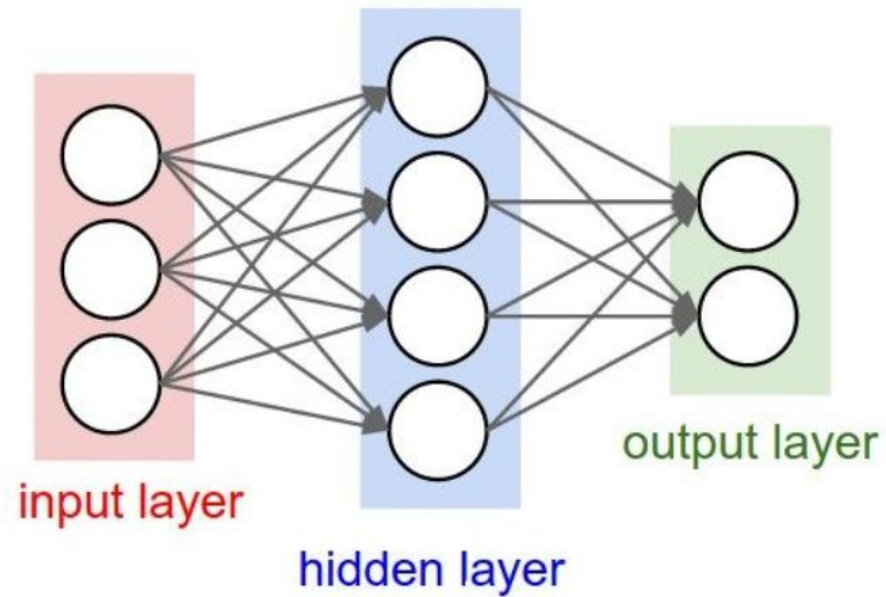
$$f = \sigma(W_2 \sigma(W_1 x))$$

- MLP de 3 capas:

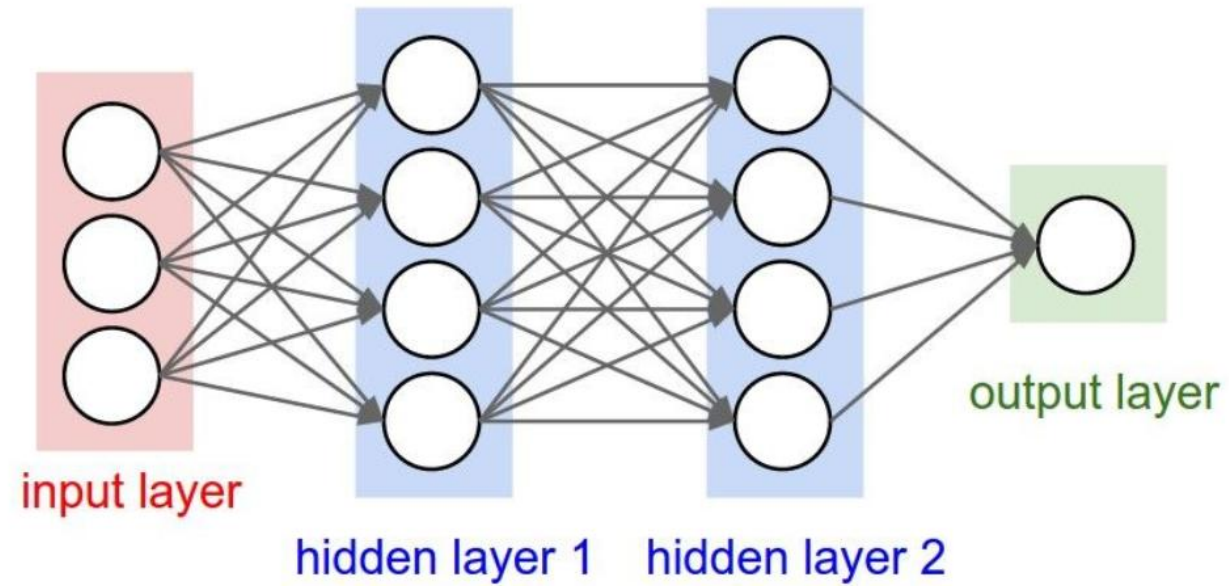
$$f = \sigma(W_3 \sigma(W_2 \sigma(W_1 x)))$$



Visualmente, representaremos los MLP utilizando capas



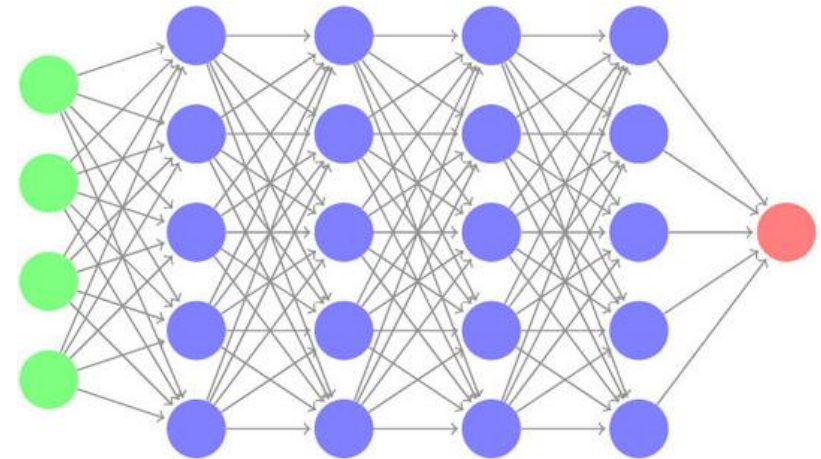
MLP (NN) de 2 capas, o NN de 1 capa oculta



MLP (NN) de 3 capas, o NN de 2 capas ocultas

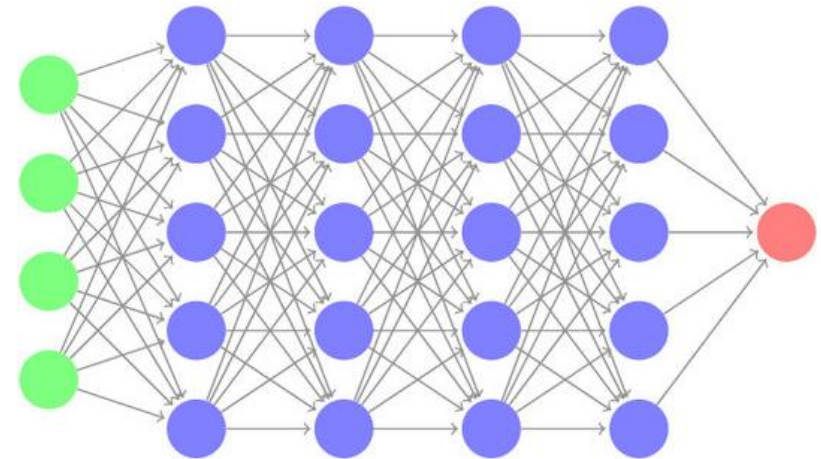
MLP generaliza el concepto de Perceptron

- Un Multilayer Perceptron (MLP), combina múltiples perceptrones en múltiples capas. Es conocido también como Deep Feed Forward Network (DFF).
- ¿Permite esta estructura resolver problemas más complejos que los que puede resolver un Perceptron? **SÍ**
- Una pregunta interesante es entonces, ¿hasta dónde podemos llegar con un MLP?



¿Hasta dónde podemos llegar con un MLP?

- De hecho, hasta donde uno quiera.
- De acuerdo al *teorema de aproximación universal**, un MLP con 1 capa oculta puede aproximar cualquier función continua, con precisión arbitraria.
- Además, un MLP con 2 capas ocultas es capaz de aproximar cualquier función, con precisión arbitraria.
- En ambos casos, la cantidad de neuronas por capa va a depender del problema, o sea, puede ser potencialmente muy grande.



*<https://www.youtube.com/watch?v=ljqkc7OLenI>

Frameworks al alcance de todos

- La implementación de todos los elementos que vimos de manera conceptual es bastante difícil en la práctica.
- Sin embargo, gran cantidad de equipos se han dedicado durante la última década a desarrollar *frameworks* y librerías eficientes y sumamente prácticas.
- Lo mejor es que (casi) todo es gratuito.
- Todo esto ha contribuido en gran medida a la popularidad de DL.



Frameworks al alcance de todos

La lista de frameworks es amplia, pero los más populares son:

- Keras: <http://keras.io/> (Google)
- TensorFlow: <http://tensorflow.org/> (Google).
- PyTorch: <http://pytorch.org/> (Facebook).
- CNTK: <https://github.com/microsoft/CNTK> (Microsoft)



Usaremos Keras sobre Google Colab

- Keras es el framework con el nivel de abstracción más alto
- Código libre, corre por sobre TensorFlow2.
- Escrita en y para Python.
- Muy fácil de usar, altamente modular.
- Foco en permitir experimentación rápida.
- Soporta (casi) todos los tipos de redes nativamente.
- Corre transparentemente en CPU y/o GPU.
- Lo mejor: es llegar y usar en Colab.



Keras es altamente modular

Cada tipo de elemento que se usan en las redes está modularizado:

- Capas (densas, *embedding*, convolucionales, *pooling*, etc).
- Funciones de pérdida.
- Optimizadores.
- Esquemas de inicialización.
- Funciones de activación.
- Y más, lo que además permite agregar nuevos elementos.



Veamos ahora con más detalle algunos elementos

- La estructura principal de Keras es un **modelo**.
- Los modelos permiten organizar los módulos que componen a una red. Es una especie de contenedor.
- Existen dos formas de instanciarlos:
 - Encadenando llamadas a funciones para definir el procesamiento de los datos

```
import tensorflow as tf

inputs = tf.keras.Input(shape=(3,))
x = tf.keras.layers.Dense(4, activation=tf.nn.relu)(inputs)
outputs = tf.keras.layers.Dense(5, activation=tf.nn.softmax)(x)
model = tf.keras.Model(inputs=inputs, outputs=outputs)
```

Veamos ahora con más detalle algunos elementos

- La estructura principal de Keras es un **modelo**.
- Los modelos permiten organizar los módulos que componen a una red. Es una especie de contenedor.
- Existen dos formas de instanciarlos:
 - Heredando de la clase *Model*

```
import tensorflow as tf

class MyModel(tf.keras.Model):

    def __init__(self):
        super(MyModel, self).__init__()
        self.dense1 = tf.keras.layers.Dense(4, activation=tf.nn.relu)
        self.dense2 = tf.keras.layers.Dense(5, activation=tf.nn.softmax)

    def call(self, inputs):
        x = self.dense1(inputs)
        return self.dense2(x)

model = MyModel()
```

La clase Sequential hace las cosas aún más fáciles

Esto crea un contenedor vacío llamado *model*

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential()
```

Sequential model container



La clase Sequential hace las cosas aún más fáciles

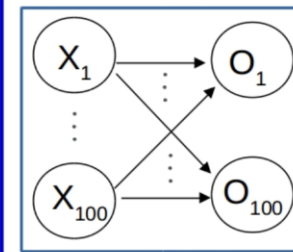
Luego agregamos un módulo que implemente una capa densa con 100 entradas y 100 salidas

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential()

model.add(keras.Input(shape=(100,)))
model.add(layers.Dense(100))
```

Sequential model container



La clase Sequential hace las cosas aún más fáciles

Luego agregamos una función de activación ReLU

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential()

model.add(keras.Input(shape=(100,)))
model.add(layers.Dense(100))

model.add(layers.ReLU())
```

Sequential model container



La clase Sequential hace las cosas aún más fáciles

Finalmente agregamos una nueva capa densa seguida de un softmax

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

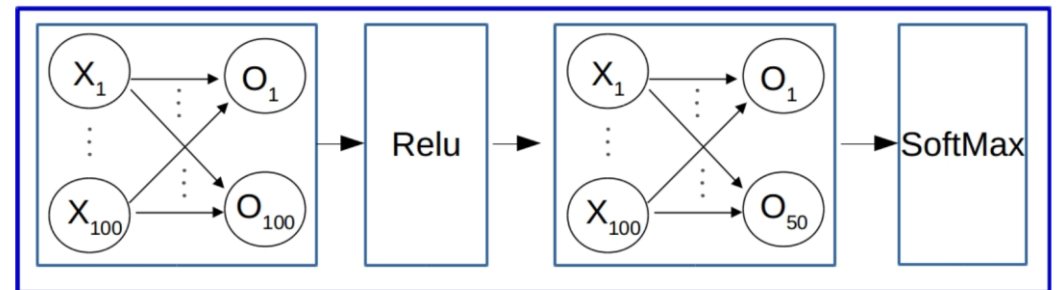
```
model = keras.Sequential()
```

```
model.add(keras.Input(shape=(100,)))
model.add(layers.Dense(100))
```

```
model.add(layers.ReLU())
```

```
model.add(layers.Dense(50))
model.add(layers.Softmax())
```

Sequential model container



La clase Sequential hace las cosas aún más fáciles

Todo lo anterior puede hacerse también en “una” línea de código.

```
model = keras.Sequential(  
    [  
        keras.Input(shape=(100,)),  
        layers.Dense(100),  
        layers.ReLU(),  
        layers.Dense(50),  
        layers.Softmax(),  
    ]  
)
```

Falta un último paso

Para poder entrenar un modelo, es necesario compilarlo previamente

```
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])  
model.fit(x_train, y_train, batch_size=128, epochs=15, validation_split=0.1)
```

Vamos a Colab...



Ejercicios propuestos

Parte 1: MLP con imágenes

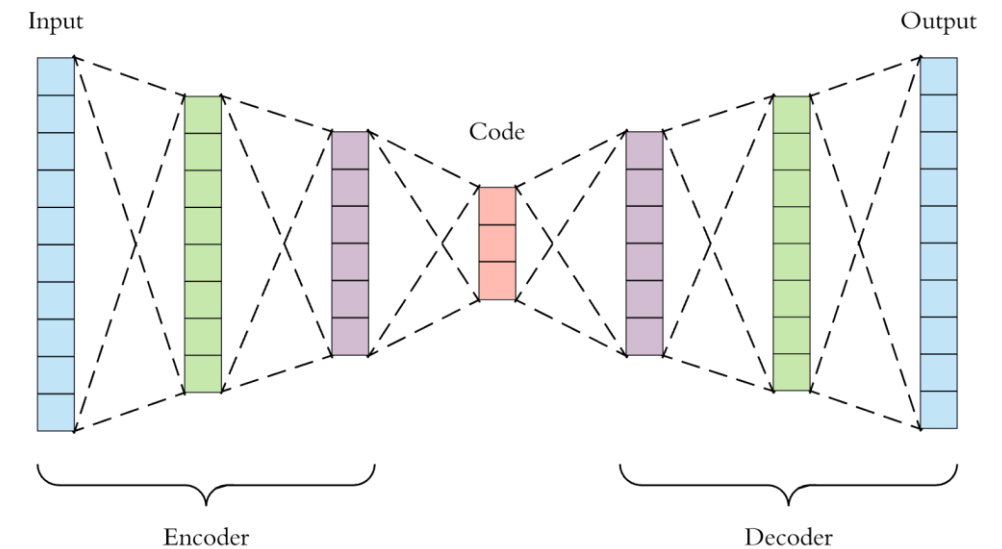
- Implemente en Keras un MLP con la cantidad de capas y neuronas que prefiera, que permita hacer clasificación de imágenes.
- Entrene la red en CIFAR10 y reporte valores relevantes: velocidad por época, pérdida entrenamiento, pérdida validación, etc.
- Juegue con los hiperparámetros y estructura de la red hasta obtener el rendimiento más alto en el set de validación/test.



Ejercicios propuestos

Parte 2: Autoencoder

- Un *Autoencoder* es una arquitectura de red neuronal que se utiliza para aprender features generativas, es decir, que permitan generar datos.
- En particular, un Autoencoder utiliza una estructura de cuello de botella para generar un output que sea lo más parecido posible al input.
- Utilizando Keras, construya y entrene Autoencoders para alguno de los sets de datos de imágenes disponibles.
- Utilice luego los códigos (**Code** en la figura) como features para entrenar un MLP sobre el mismo set de datos.
- Compare estos resultados con los de un MLP entrenado directamente desde las imágenes.



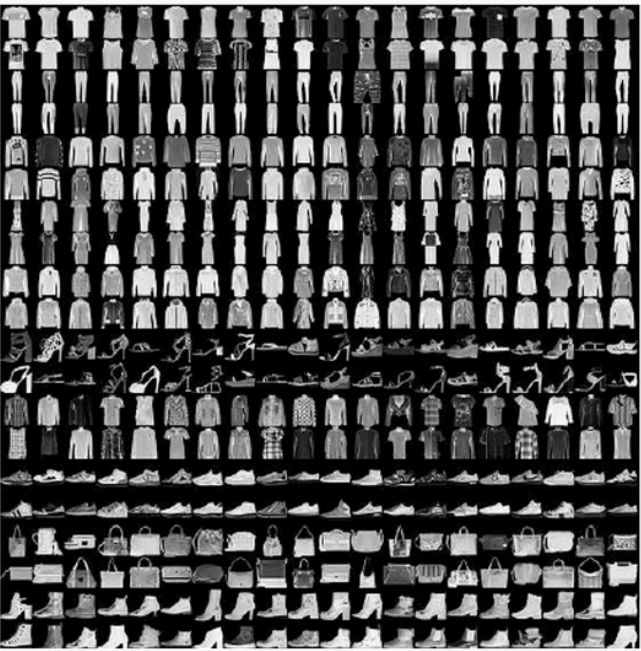
Recursos recomendados para el curso

- “Deep Learning”, de Goodfellow y colaboradores: <https://www.deeplearningbook.org/>
- “Dive into Deep Learning”, de Zhang y colaboradores: <https://d2l.ai/>
- “Deep Learning with Python”, 2ª ed., de F. Chollet.
- Ejemplos y tutoriales oficiales de Keras

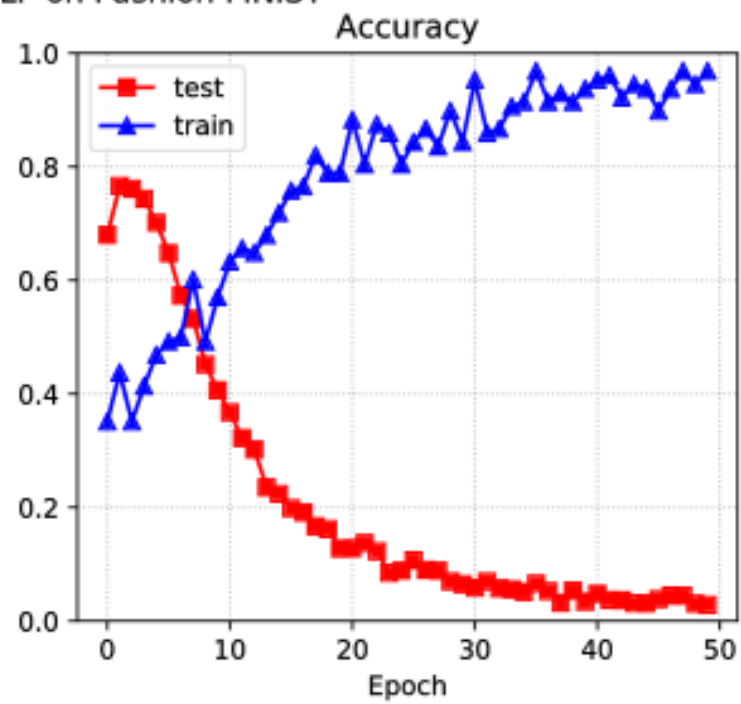
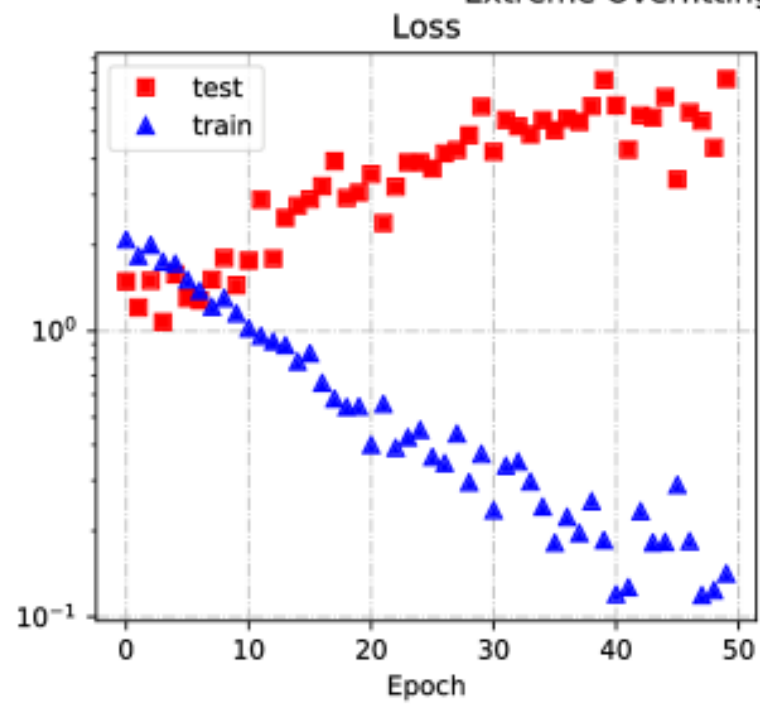
Y ahora, el gran número de cierre

El set de datos **Fashion MNIST** permite construir **clasificadores de ropa** a partir de imágenes

- Sucesor de MNIST con mayor dificultad (pero tampoco tanto más).
- Cada imagen muestra una sola prenda de una de diez categorías. Las imágenes son binarias con una resolución de 28x28 píxeles.
- El *dataset* consta de 60.000 ejemplos de entrenamiento y 10.000 de test.
- ¿Qué pasa si usamos el mismo tipo de MLP que usamos para MNIST?

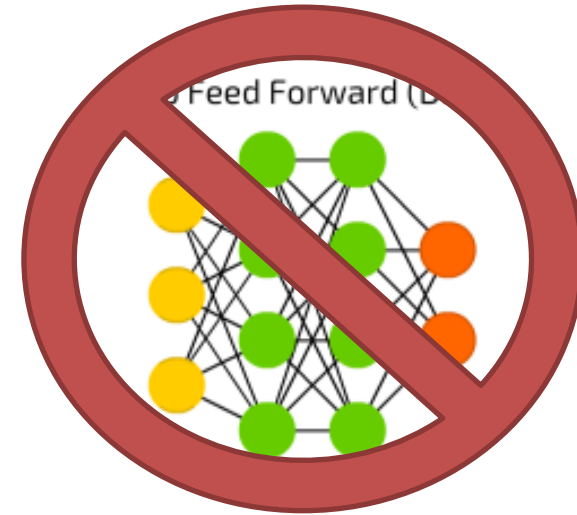
Label	Description	Examples
0	T-Shirt/Top	
1	Trouser	
2	Pullover	
3	Dress	
4	Coat	
5	Sandals	
6	Shirt	
7	Sneaker	
8	Bag	
9	Ankle boots	

Extreme Overfitting MLP on Fashion MNIST



Los MLP pueden ser muy difíciles de hacer funcionar exitosamente

- MLP tienen tanto poder, que memorizan hasta el ruido en los datos (¿es el mínimo global siempre el mejor?).
- Interpretación de los modelos es altamente compleja.
- La próxima clase veremos formas de solucionar este problema.



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



Deep Learning

Introducción a Deep Learning

Ariel Reyes Pardo
Dpto. Ciencia de la Computación